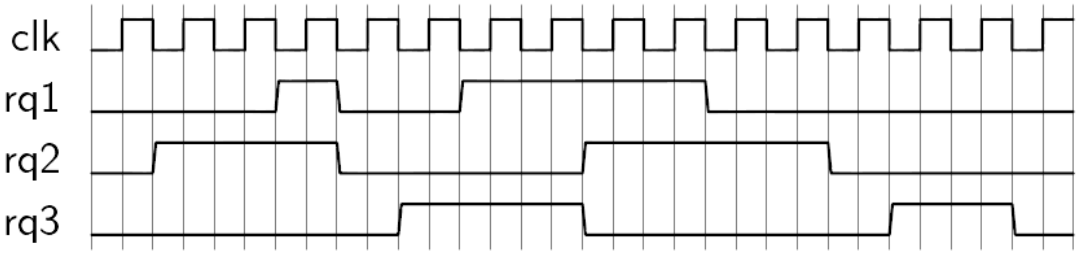
CE2003 Tut 2

Q1 A simple arbiter circuit has three request inputs, **rq1**, **rq2**, and **rq3**. It generates three grant outputs, **gt1**, **gt2**, and **gt3**. Only one grant output can be high at any point in time. When a request input is asserted while in the *idle state*, the circuit should grant that request by asserting the corresponding **gt** output. This should remain high until the corresponding **rq** input is de-asserted, at which point the circuit should move to the idle state. If multiple **rq** inputs are asserted, the highest priority request is granted, with a priority order of **rq1** (highest) to **rq3** (lowest).

(a) Draw a state transition diagram to capture this behaviour.

(b) Implement this state machine in Verilog using separate always blocks for the state transition and state register blocks.

(c) Write a Verilog testbench that will test the following input sequence, and draw the expected outputs. Assume a 10ns clk, and assume that rst is applied for 10ns prior to the signals below and is then negated.



Q2 You are provided with a 6-bit unsigned adder module with the following declaration:

```
module adder6b (input [5:0] a, b, output [5:0] sum);
```

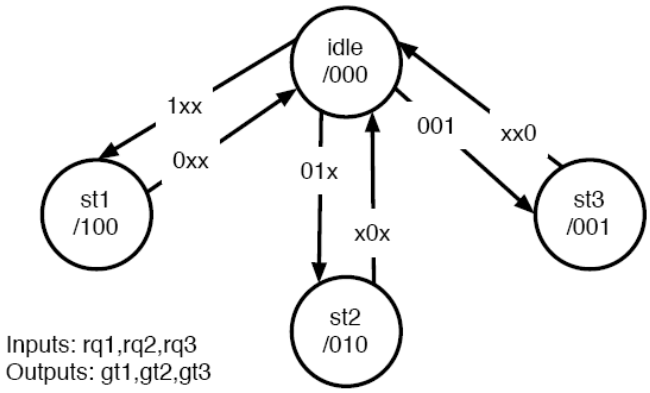
Write a self-checking testbench that tests the module with 10 random inputs, and displays a message, showing the incorrect value and expected result, if an error is detected. You can use a repeat loop as follows:

```
repeat (num_times)
begin

end
```

Answer 1

(a) The following state transition diagram captures the required behaviour. Note the don't cares on the transitions which enforce the required priority.



(b) The following code implements the above state machine:

```
module arb_fsm (input clk, rst, rq1, rq2, rq3,
                    output reg gt1, gt2, gt3);

reg [1:0] st, nst;
parameter IDLE = 2'b00, ST1 = 2'b01, ST2 = 2'b10, ST3 = 2'b11;

always @*
begin
nst = st;
gt1 = 1'b0; gt2 = 1'b0; gt3 = 1'b0;
case (st)
    IDLE:
        if (rq1) nst = ST1;
        else if (rq2) nst = ST2;
        else if (rq3) nst = ST3;
    ST1: begin
        if (~rq1) nst = IDLE;
        gt1 = 1'b1;
    end
    ST2: begin
        if (~rq2) nst = IDLE;
        gt2 = 1'b1;
    end
    ST3: begin
        if (~rq3) nst = IDLE;
        gt3 = 1'b1;
    end
endcase
end

always @(posedge clk)
begin
    if(rst) st <= IDLE;
    else st <= nst;
end

endmodule
```

(c) We can use the Verilog delay syntax to recreate that input pattern. We will also add a rest pulse prior to that input sequence.

```
module fsm_tb;

reg clk, rst, rq1, rq2, rq3;
wire gt1, gt2, gt3;

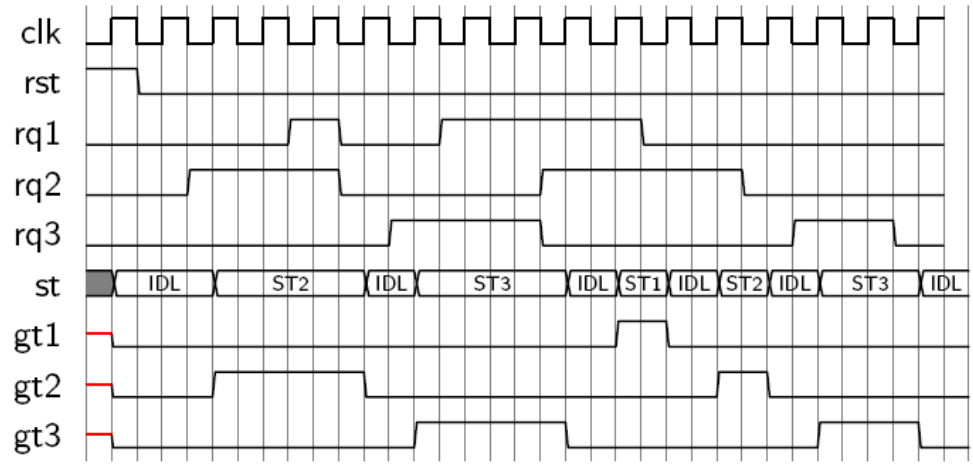
arb_fsm uut (.clk(clk), .rst(rst), .rq1(rq1), .rq2(rq2), .rq3(rq3),
             .gt1(gt1), .gt2(gt2), .gt3(gt3));

initial
begin
    clk = 1'b0; rst = 1'b1;
    rq1 = 1'b0; rq2 = 1'b0; rq3 = 1'b0;
    #10 rst = 1'b0;
    #10 rq2 = 1'b1;
    #20 rq1 = 1'b1;
    #10 rq1 = 1'b0; rq2 = 1'b0;
    #10 rq3 = 1'b1;
    #10 rq1 = 1'b1;
    #20 rq2 = 1'b1; rq3 = 1'b0;
    #20 rq1 = 1'b0;
    #20 rq2 = 1'b0;
    #10 rq3 = 1'b1;
    #20 rq3 = 1'b0;
    #10 $finish;
end

always #5 clk = ~clk;

endmodule
```

This results in the waveform shown:



Answer 2

We use a repeat loop with a delay to try multiple values. The `$random` function gives us a 32-bit random number so we can use it to give us the 6-bit inputs.

```
module adder_tb;

reg [5:0] a, b;
wire [5:0] sum;

adder6b uut (.a(a), .b(b), .sum(sum));

initial
begin
    repeat(10) begin
        {a,b} = $random;
        #10 if (sum !=  a + b) $display("Error %0d + %0d should be %0d, not %0d", a, b, a+b, sum);
    end
    $finish;
end

endmodule
```

Note the line `{a,b} = $random;` generates the 32-bit random number then assigns it to a 12-bit concatenation of `a` and `b`. So `a` will get bits 11 to 6 and `b` will get bits 5 to 0.